

Дизайн рекомендательного сервиса

ШАД, Рекомендательные системы
2025/26, лекция 9

Даня Ткаченко

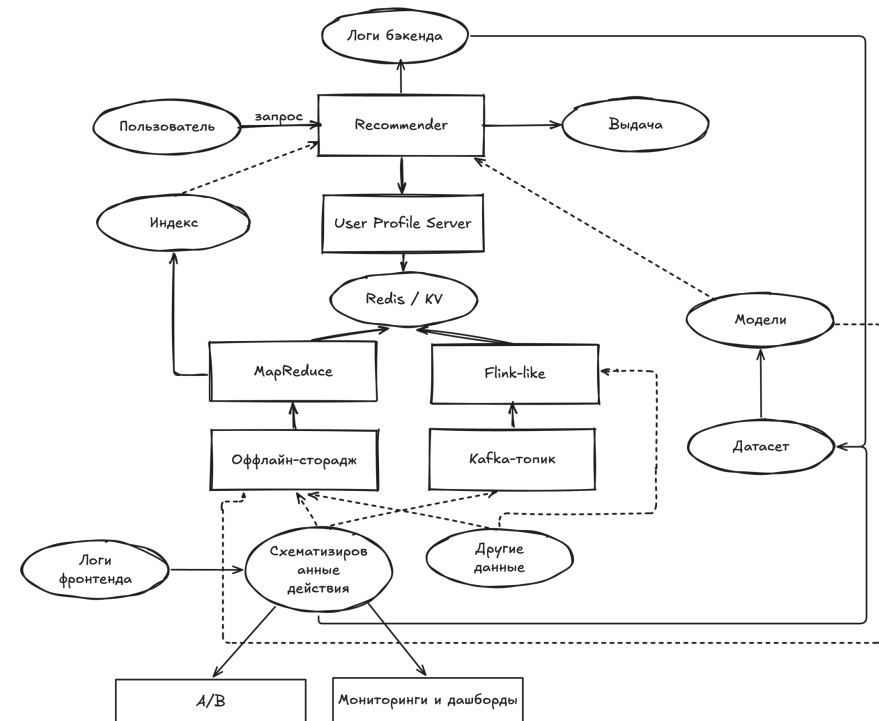
Руководитель службы ML-сервисов,
Яндекс Лавка

Яндекс



На чем остановились в прошлый раз

- На прошлой лекции мы посмотрели на архитектуру всей рексистемы
- Сегодня более плотно посмотрим на архитектуру конкретно рекомендера — ее главной части



План занятия

На лекции:

1. Архитектура рекомендательного сервиса
2. Практические трюки
3. Устойчивость рексистемы

На семинаре:

- Разберем статью от LinkedIn с их подходом к кандидатогенерации и фильтрации – LiNR

Архитектура рекомендательного сервиса

Часть 1

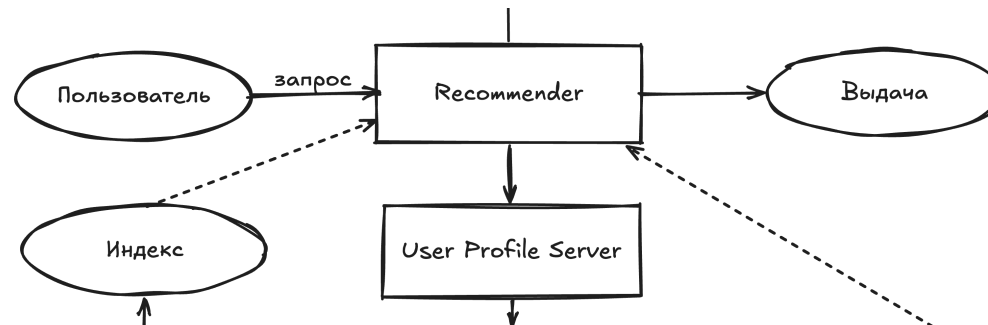


Напоминание

- Вспомним флоу типичной рекомендательной системы (справа)
- Давайте поймем, как этот флоу на практике имплементируется в рантайм-сервисы
- Но перед этим: зачем это мне, если я занимаюсь МЛ для рекомендаций?



Базовый вариант



- На сервисе живет индекс айтемов, обновляется раз в сутки
- В индексе лежит все необходимое для расчета моделей
- Запрос прилетает, мы идем в наш сервис профиля пользователя, получаем что-то в запросе
- Поднимаем кандидатов
- Считаем для кандидата фичи*
- Считаем модельку
- Ранжируем и отдаем ответ

Узкие места?

- Обновлять информацию об айтемах раз в сутки для некоторых продуктов нереалистично, например, новости
- Один сервис делает всю работу: могут быть сложности с параллельностью и устойчивостью, может превратиться в легаси-монолит, сложность для работы в большой команде
- Обычно мы хотим отвечать за что-то порядка 50-300мс в зависимости от продуктовых требований, если айтемов много, просто положить их в память на одну машинку будет сложно, а читать из диска много данных в рантайме — самоубийство

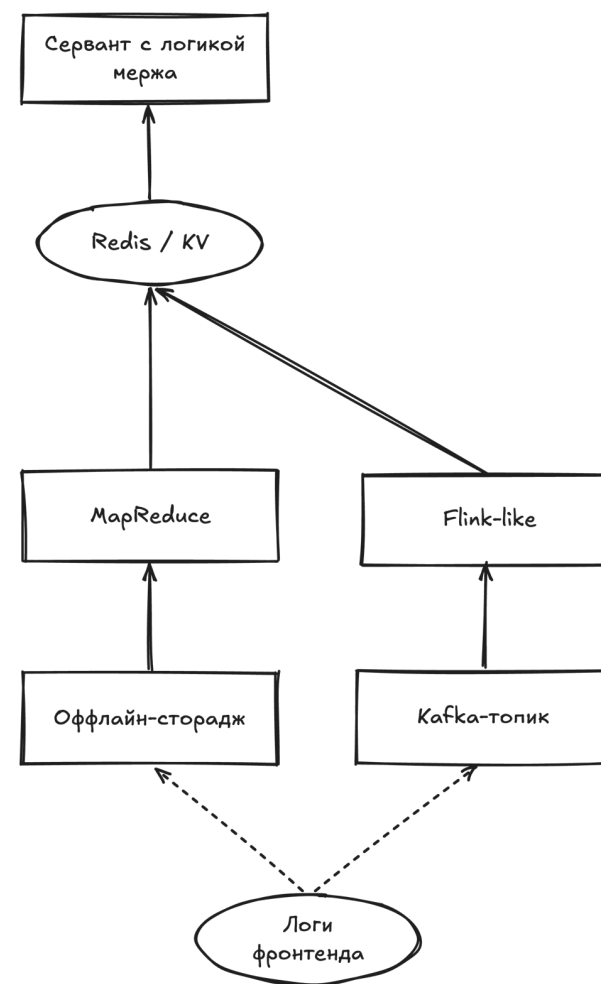
Давайте глянем на все по очереди

Динамический индекс

- Иногда нужен динамический индекс айтеров
- Что происходит при появлении нового айтера:
 - Появился айтер, проехал по кафкам, получили
 - Название, описание, картинку
 - После этого контентный эмбединг
- Дальше едет по кафке в отдельный индекс или сервис, который потом используется в кандидатогенерации

Что с айтемными статистиками?

- Просто становится частью Feature Storage'a
- E.g. используем лямбда-архитектуру



Практический путь работы со свежими айтемами:

- Основной сервис → отдает какие-то айтемы
- Свежий сервис → отдает какие-то айтемы (только свежие)
- Блендинг-сервис замешивает контролируемым способом

Вопрос от внимательного слушателя:

Зачем вообще айтемам ехать в свежий сервис? Если мы их фичи все равно храним в лямбде

Работа свежего сервиса

- В более сложном варианте на свежем сервисе происходит прямо отдельное ранжирование
- В более простом — он по сути он выполняет роль источника свежих кандидатов
 - используют trending-топы
 - просто случайных кандидатов
 - апп-ы
 - иногда i2i-списки

В любом случае для ответа тут нужно ответить на вопрос — почему кандидатогенерацию выносят в отдельный сервис, ответ будет тем же. Плавно переходим к следующему пункту

Кандидатогенерация aaS

На практике в большой рексистеме мы увидим, что кандидатогенерацию (в первую очередь) и подсчет фичей (во вторую очередь) вынесут в другие сервисы

Почему так происходит с кандидатами?

- Ключевое и самое главное тут — этой другой паттерн нагрузки
- Фичи и модели — cpu/gpu-bound (фичи отчасти memory bound)
- Кандидатогенерация — memory-bound
- Другой паттерн использования памяти относительно фичей
- Тяжело скейлить такой сервис горизонтально, ему одновременно надо много и памяти, и cpi

Кандидатогенерация aaS

Другие причины

- Separation of concerns
- Минус SSO
- Позволяет гибче экспериментировать, не затрагивая другой сервис
- Естественное место для покомандного разделения

Проблемка -- фичи отдельно от кандидатов:

- на практике может быть дублирование каких-то данных по памяти
- на практике все равно на это идут

Фичи aaS

Почему фичи вывозят в отдельный сервис тоже

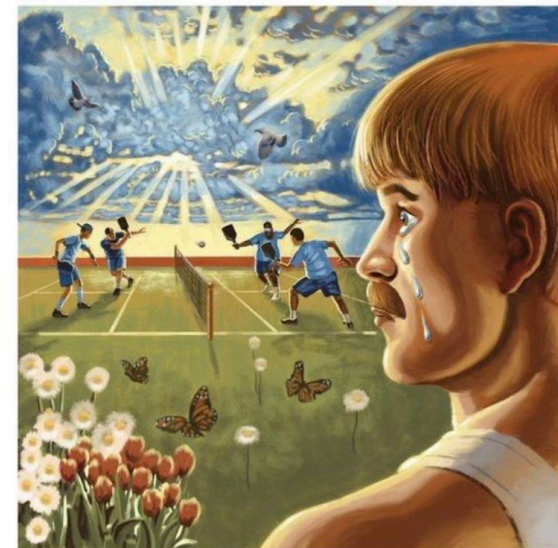
- Инкапсуляция логики Feature Store'a
- Фичи все еще больше memory bound относительно сри-гри, чем подсчет моделей
- Организационные причины те же, что и для кандидатов

The New York Times

OPINION
MICHELLE COTTLE

Is the Cure to Male Loneliness building B2B SaaS with the homies?

July 19, 2023



Benjamin Marra

Фичи aaS

Почему фичи вывозят в отдельный сервис тоже

- Инкапсуляция логики Feature Store'a
- Фичи все еще больше memory bound относительно сри-гри, чем подсчет моделей
- Организационные причины те же, что и для кандидатов

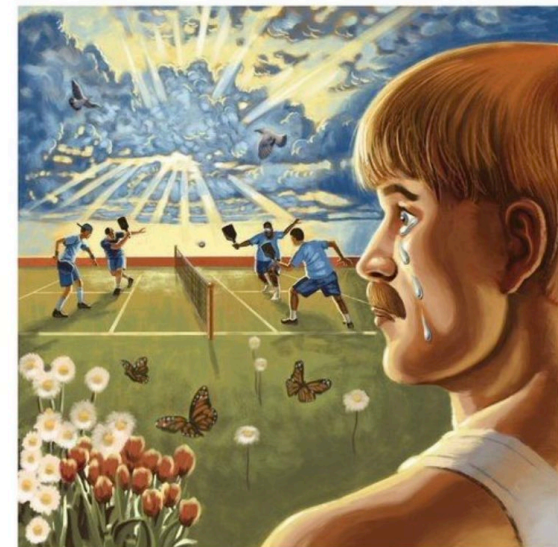
Иногда тем не менее бывает, что никакой памяти не хватает, даже если мы разнесли все по нескольким сервиса. Какие у нас варианты?

The New York Times

OPINION
MICHELLE COTTLE

Is the Cure to Male Loneliness building B2B SaaS with the homies?

July 19, 2023



Benjamin Marra

Вариант 1: префильтрация

Выбрать только какое-то подмножество айтеров, которое вообще сможет рекомендоваться на уровне продуктовых или других ограничений

- Например, разрешить попадать в рекомендации только относительно новым айтерам (так делают, например, в новостях)
- Или предиктить неперсональный профит айтера и по нему отрезать то, что попадет потом в индекс (так делают много где)

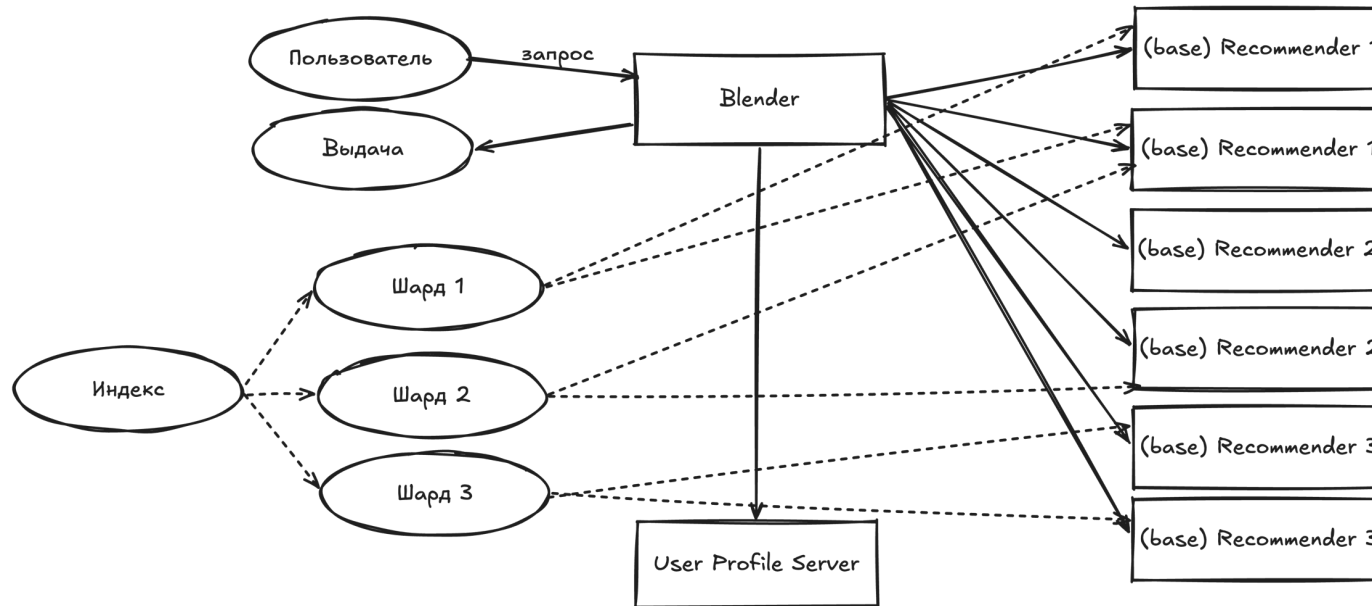
Вариант 2: шардирование

Шард (англ. shard — осколок) — индекс, построенный на подмножестве айтеров. Разбиваем множество всех айтеров на X кусков, внутри каждого строим по сути независимую систему.

Другая возможная причина для шардирования: мы просто хотим скорить больше кандидатов. Почему оно помогает? Давайте глянем на схему

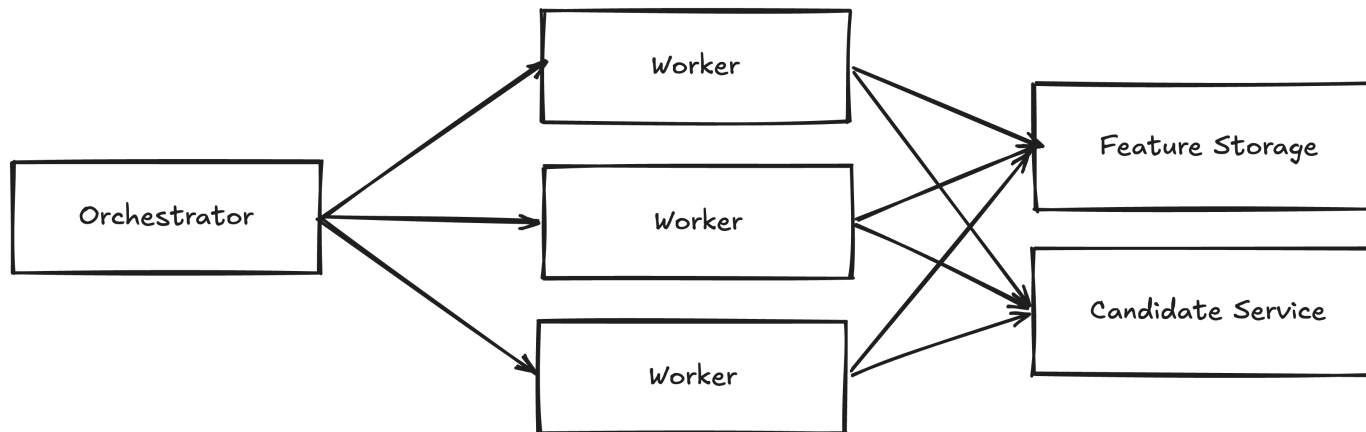
NB: на практике шардирование чаще всего используется одновременно с прошлым вариантом

Шардирование в простом случае



- У нас появляется какой-то сервис над шардами, который нам необходим, чтобы объединять их результаты вместе.
- На нем логично расположить более тяжелый этап ранжирования, если у вас такой есть.
- Обычно его называют блендером, средним или еще как-то так

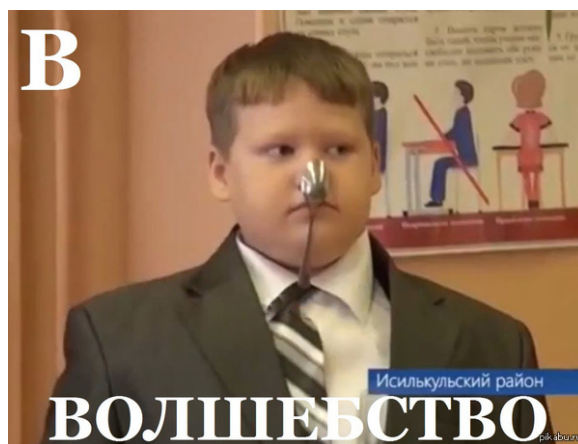
Шардирование в более сложном случае



- В более сложном случае у нас шардируются конкретно feature-storage и candidate-сервис.
- Инференс-сервер, который работает с ними шардировать нет смысла, но можно использовать несколько параллельных для ускорения.
- Тогда мы получаем схему с оркестратором, воркер-сервисом, кандидат-сервис, фиче-сервисом

Двигаемся дальше

Мы немного поговорили о том, как в целом может быть устроен рекомендательный сервис, или сервисы, как мы теперь видим. Давайте посмотрим на пару полезных трюков, которые могут пригодиться в работе



Практические трюки

Часть 2



Нюанс

Мы до этого говорили про шарды так, как будто нам всегда нужно сгенерировать кандидатов по абсолютно всем айтемам из них. Но на практике это не так:

- Некоторые товары (например, 18+) показывать нельзя никогда
- Некоторые товары нельзя показывать конкретно этому пользователю (тип подписки не совпадает с типом контента, если это кинотеатр)
- Некоторые товары нельзя показывать временно (товар отсутствует на складе)
- Часто бывает нужен механизм мгновенного бана айтемов в рекомендациях

Все это приводит нас к выводу, что нужно уметь фильтровать айтемы в рантайме.

Где фильтровать?

Правильный ответ: сразу после кандидатогенерации

Почему: если пофильтровать не там

1. Потратим дорогое инференсное время на айтемы, про которые мы заранее знаем, что не покажем их. Могли бы потратить их на других кандидатов, выдача была бы строго не хуже.
2. Выдача может быть вообще короткой, что определенно приведет к падению бизнес-метрик. Либо придется делать дозапросы, что увеличит общее время ответа системы.



Как фильтровать?

Общая проблема: мы хотим поддерживать систему, где мы гарантируем, что ранжируется ровно X кандидатов. Наши варианты:

1. Всегда набирать кандидатов с запасом
2. Устроить саму кандидатогенерацию так, чтобы кандидаты генерировались уже пофильтрованными

На самом деле тут та же проблема, что и на предыдущем слайде, и понятно, что вариант 2 предпочтителен.

Как фильтровать?

Глобально 2 типа ограничений:

1. Глобальная доступность/недоступность
2. Доступность контента бьется на X кластеров. Где можем, поддерживаем легкие фильтрации на основе чего-то типа блумфильтров (про них еще дальше) или просто bitset-ов. Но иногда строим X копий кандидатогенератора (HNSW).

Тут на семинаре посмотрим на LiNR от LinkedIn: про то, как обходят эту проблему в их exact-KNN системе кандидатогенерации на GPU.

- Мы тут настроили кучу сервисов, выглядит, что данные бесконечно летают по сети туда-сюда.
- Нет ли с этим проблем?
- Конечно, есть. Ответ как всегда в кешировании.

Простой случай: поиск

- Если у нас неперсональный поиск, то кешировать можно прям весь ответ 🔥
- Если персональный, то скорее всего где-то есть неперсональная часть, и можно кешировать ее.

Например, в лавке: модель релевантности → модель покупки. Модель релевантности можно кешировать (но на практике у нас еще чуть более хитрое кеширование) на какой-то TTL.

Правило безопасности при работе с кешами, заслуживающее отдельного слайда: случайно не инвалидируйте весь кеш, уложите сервис.

Для этого:

- а) бережно выбирайте ключи кеширования
- б) сделайте на это отдельные тесты.

Более общий случай: что кешировать?

Тут понятно, что вопрос у нас по сути продуктовый, при этом есть один базовый безопасный вариант.

- айтемные фичи (с разумным TTL)

Другие кандидаты для кеширования:

- можно устроить кандидатогенерацию так, чтобы одна часть явно соответствовала долгосрочным интересам пользователя, кешировать ее
- кешировать всю выдачу, но инвалидировать кеш об специфические действия (типа добавления в корзину).
- на что вы еще уговорите вашего продуктового менеджера

Про метаинформацию

В этом же месте хочется сделать оговорку
Наш рекомендательный сервис выдает просто
упорядоченный список идентификаторов айтемов. На
выдачу айтемы попадают уже со стопкой
метаинформации: картинка, название, текст, описание,
что-то еще.

Этого обычно добиваются 2 путями:

- хороший: поверх живет отдельный сервис, который этим занимается.
- исторический: специальный вид запроса в рекомендательную систему, который просит вернуть мету для нужных айтемов (для яндексоидов: сниппетный запрос). произрастает из соблазна переиспользовать для этого фиче-сторадж.

Второй путь открывает некоторые дополнительные возможности, но делать так все же обычно не стоит, если ваши разработчики не супергерои, будет тяжело in the long run.

Пагинация

Отдельно упомянем отдельный жанр фильтрации, для которого есть элегантный технический трюк: ранжирование под пагинацию.

- Обычная идея пагинации: сначала показали первые X товаров, потом показали вторые X товаров
- Проблема: рекомендательная система не стабильно, наивная имплементация может вести к дублям.

Вариант решения:

- Будем бросать между фронтом и бэком между пагинациями в отдельном параметре сериализованный (например, фильтр Блума) объект с тем, что было отдано с самой первой пагинации.

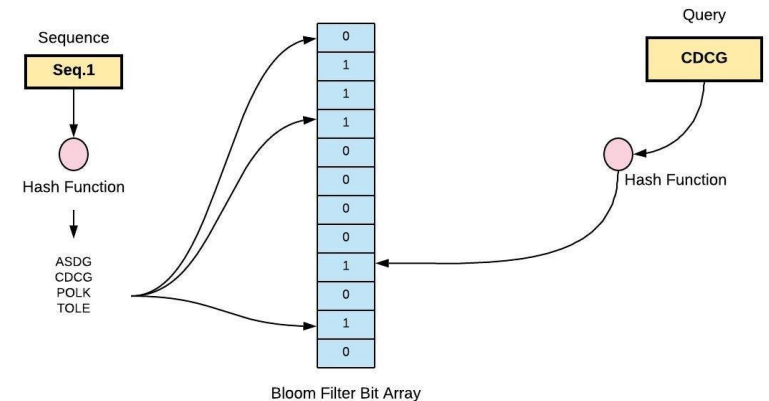
Напоминание: фильтр Блума

Структура данных, предоставляет интерфейс

- `insert()` → вставить элемент за $O(1)$
- `maybe_has()` → $O(1)$
 - возвращает, есть ли элемент внутри
 - если ответило нет, айтема точно нет
 - если ответило да, айтема на самом деле все же может не быть

Примерное устройство

- К независимых хешфункций с образом $[0, M - 1]$
- Массив длины M из нулей-единиц
- Когда добавляем элемент, в элементы массива, полученные из функций, ставим 1
- Когда проверяем, смотрим на те же значения, если везде 1, то говорим, что элемент есть



Для справки вероятность ошибки примерно

$$(1 - e^{-kn/m})^k$$

где n — сколько уже вставлено элементов

Фильтр Блума вообще очень часто используется для фильтраций, полезно представлять, что это.

Про блендинг

Мы уже несколько раз упоминали блендинг

- по большей части в контексте мержа данных с разных шардов
- или мержа данных с разных кандгенов
- но чаще всего на этом ничего не заканчивается

Иногда бывает важно уметь гарантировать фиксированную долю определенных айтемов в выдаче. Или поддерживать какую-то другую статистику нашей выдачи на определенном уровне.

Простой пример:

- Реклама
- Квота под экслорейшн
- Уровень разнообразия

Про блендинг

Простой вариант, которым многие пользуются – PID-контроллер.

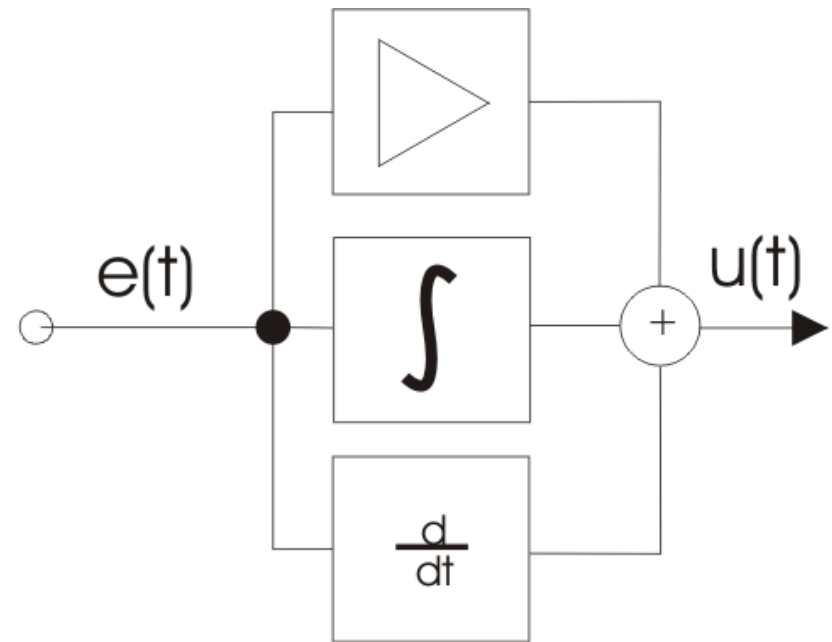
$$u(t) = P + I + D = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de}{dt},$$

Идейно:

P: Реакция на **текущую** ошибку. Чем больше разница между целью и фактом, тем сильнее воздействие.

I: Реакция на **накопленную** ошибку. Чтобы мы действительно добежали до нужного таргета, а не просто рядом с ним бегали.

D: Реакция на **скорость изменения** ошибки. Для предотвращения перерегулирования.



Устойчивость рексистемы

Часть 3



Самое главное правило надежной рексистемы

Самое важное правило надежной рексистемы:

- она не должна умирать героически
- она должна деградировать предсказуемо

Возможные проблемы:

- умер кандидатный сервис
- недоступен feature storage
- не подгрузилась модель
- один из шардов отвалился
- данные устарели
- все жив, но не укладываемся в SLA

Во всех этих случаях 500ить – точно худший вариант

Давайте глянем, что с этим обычно делают

Лестница фоллбеков

Пример лестницы фоллбеков

- Полный персональный сценарий
 - Работают все сервисы, все отлично
 - Тут мы должны находиться большую часть времени
- Блендеру плохо
 - Автоматически отключаем поздние нейросетевые переранжирования
 - Не помогает -> берем меньше кандидатов
- Плохо сервису кандидатогенерации
 - Поддерживаем совсем простую экстренную кандидатогенерацию по ANN / популярности на блендере по небольшому набору айтемов
- Плохо сервису фичей
 - Отвечаем скорыми ANN
- Все таки не отвечаем совсем
 - Показываем тыкву

Важно:

- переходы между ступенями лучше продумать заранее
- а не рожать их в момент инцидента
- (хотя все бывает)

Тыквы

- Тыква — это намеренно не самый интеллектуальный, но точно работающий режим системы
- Например:
 - предрассчитанные (один раз!) популярные товары
 - популярное по сегменту
 - редакторские подборки
 - простая бизнес-логика без персонализации

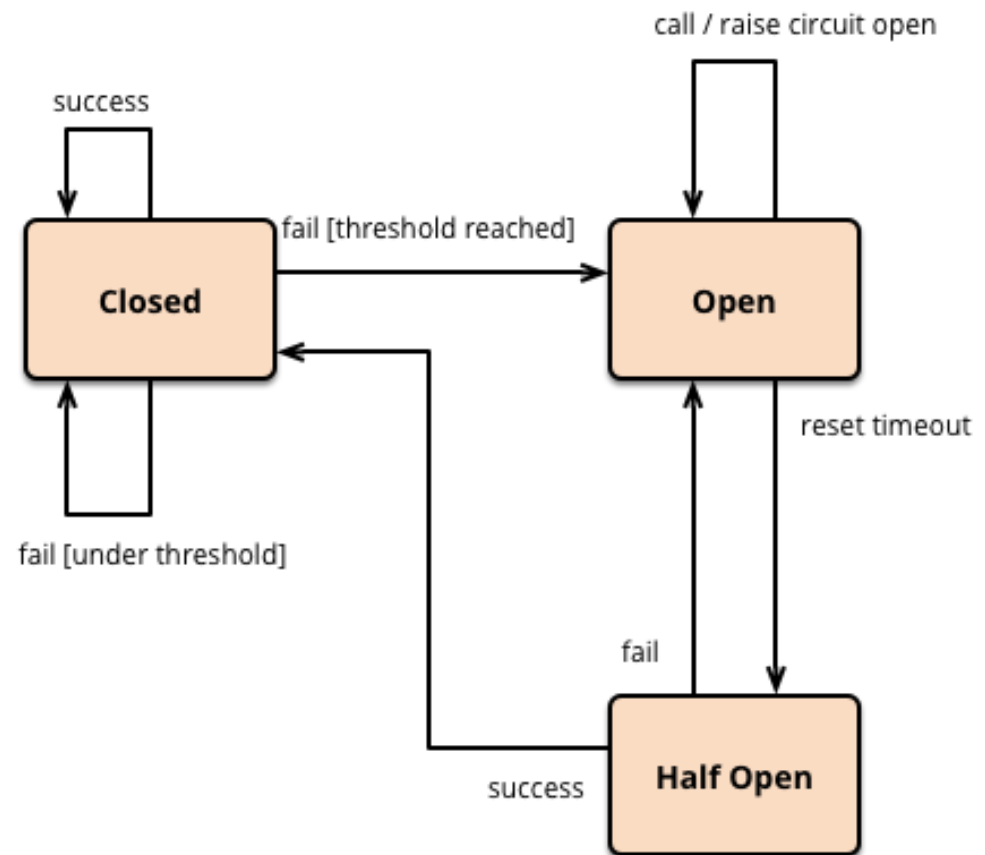
Важный момент:

- Чем тупее ваша тыквы, тем лучше, нет ничего менее приятного, чем в решающий момент узнать, что тыква не работает
- Лучше где-то на случай жесткого инцидента просто законфигить какой-то огромный список айтемов, чтобы точно пролетели все фильтрации
- Либо иметь несколько последовательных тыкв

Отсюда мысль: если вы давно не тестировали свою тыкву, значит, у вас ее скорее всего уже нет

Автоматическая деградация

- Кроме придумывания самих fallbackов нужно научиться правильно их включать
- Что обычно помогает:
 - Circuit breaker'ы
 - привязать все к утилизации сри или что-то в таком духе
 - делать тыквы снаружи от вашего сервиса (the golden rule)



Что мониторить в рексистеме?

Обычные технические метрики:

- latency p50 / p95 / p99
- error rate
- timeout rate
- cache hit rate
- лаги очередей

Специфичные для рекомендаций штуки:

- Какие?



уже было в прошлый раз, но не могу удержаться

Что мониторить в рексистеме?

Специфические для нас:

- доля разных кандидатогенераторов в выдаче
- доля свежих айтемов
- доля fallback-ответов по разным фоллбекам
- покрытие выдачи/всей сессии разными категориями
- возраста модели / индекса / фичей

С прошлой лекции:

- missing rate по фичам
- driftы распределений

Есть хорошая абстракция, которой все пользуются при построении этих мониторов, и вы ее уже видели

Что мониторить в рексистеме?

Полезно иметь мониторы не только на финальную выдачи, но и на все ее этапы (e.g. воронку кандидатов)

То есть буквально:

- сколько кандидатов подняли
- сколько выжило после фильтрации
- сколько поскорили
- сколько потом попало в ответ
- сколько времени занял каждый этап

Если в вашей рексистеме этапов больше, лучше сделать что-то такое для каждого этапа

Если у вас есть специфические блендинги, то доли по ним



Что мы узнали сегодня

- хороший рантайм рекомендаций должен хорошо работать и на уровне модели, и на уровне архитектуры
- на практике это обозначает правильные
 - декомпозицию сервисов
 - работу с памятью и сетью
 - фильтрацию и кеширование
 - контролируемый blending
 - продуманную деградацию
 - внятные мониторинги
- понимание архитектуры вашего рекомендательного сервиса – часто ключ к тому, чтобы понять, где нанести профит в следующий раз



tg: @deadpadre

Спасибо!

Даня Ткаченко,
Руководитель службы ML-сервисов Яндекс Лавки

Белград 2026

